



The GitHub Copilot App

Running a Fleet of Agents

From one Copilot collaborator to a fleet of autonomous agents — managed from one place

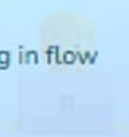
"How do we run five agents in parallel without losing control of the codebase?"

Most teams treat Copilot as one assistant at a time — but the backlog doesn't move one task at a time.

The fleet model changes the unit of work from task-I'm-doing to task-I'm-delegating.

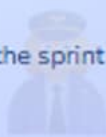
Senior Engineers

Dispatch routine work while staying in flow on architecture decisions



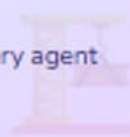
Tech Leads

Coordinate a fleet of agents across the sprint backlog from one view



Engineering Managers

Set autonomy policy and audit every agent action across the org



Serial bottleneck

Teams with one agent finish one task at a time — throughput is capped at one

5 issues, 1 morning

Five agents can process five independent tickets before the first standup

PR ready in 20 min

CVE patch and regression tests — Plan mode, two agents, full audit trail

PART 1

The Fleet Model

What agent-native means at team scale

Dashboard, dispatch table, and the decision tree for when to reach for the fleet

PART 2

Safe Parallelism

Worktrees, sandboxes, and branch isolation

How five agents work concurrently without touching each other's files

PART 3

Control Surfaces

Modes, Canvas, and the issue brief

The dial, the handbrake, and the quality lever — all five control slides

PART 4

Sessions to Systems

Repeatable workflows and Agent Merge

CVE response, workflow anatomy, and the Monday dispatch that runs itself

Click any section to jump directly there

The Fleet Model

The coordinator shift: from one agent at a time to a backlog dispatch queue



My Work Dashboard

All sessions, PRs, CI, and automations
in one coordinator view



Dispatch Table

Four agents, four issues, all running
simultaneously this morning



Decision Tree

When one task → VS Code; two or
more → the fleet model

Five agents, five issues:

↳ zero conflicts – parallel is safe by design

Already Using Copilot in VS Code? Here's the Fork

VS Code Agent Mode

Best For

Single focused task in active codebase

Session Count

One session at a time

Environment

Developer's working directory

Automation

Per-session, manual trigger

Dashboard

Chat panel inside the editor

Already in VS Code via extension

GitHub Copilot App

Best For

Parallel fleet management, multi-repo

Session Count

Multiple concurrent sessions

Environment

Isolated worktrees, cloud sandboxes

Automation

Repeatable workflows, Agent Merge

Dashboard

My Work unified view across repos

Standalone desktop install required

1 → 5

5x

20 min

My Work — The Coordinator View



Active Sessions

Task, elapsed time, mode, and current agent status
— across all repos



Open PRs

Every session-generated PR in one view; no tab-switching to check status



Background Automations

Last and next run timestamps for scheduled workflows



Notifications

Alerts when an agent needs input, hits a blocker, or opens a PR

Four Agents, Four Issues, One Morning

The dispatch table — four independent tasks running simultaneously

Agent 1	Fix accessibility violations in checkout — fixes reviewed before apply	Plan
Agent 2	Update npm dev deps — PR with package.json and lock file	Autopilot
Agent 3	Generate unit tests for auth/ — PR per exported function	Autopilot
Agent 4	Triage untagged issues — labels and priorities ready to confirm	Interactive

All four run simultaneously. Each produces a reviewable artifact — plan, PR, or action list.

When to Reach for the Fleet Model

Stay in VS Code

One active task

Single focused work in your codebase — no need to switch apps

Tight real-time collab

Same file set, interactive edits with teammates

Free tier

Technical preview requires Pro, Pro+, Business, or Enterprise

Open the Copilot App

Two or more independent tasks

Parallel dispatch is the native unit of the fleet model

Multi-repo coordination

Single view across all connected repositories

Recurring automation

Convert sessions into scheduled or event-triggered workflows

Safe Parallelism

Five agents, five worktrees — isolation by construction, not coordination



Worktree Isolation

One branch, one directory per agent —
no collision possible



Sandbox Containment

Cloud or local runtimes, bounded
scope, no lateral movement



Policy Control

Org admins configure sandbox types
and Agent Merge eligibility

```
git worktree add -b agent-session-2 ../repo-agent-2 origin/main  
↳ five isolated directories, one shared history
```

Trust by Design — Three Safety Layers



Worktree Isolation

One branch, one dir per agent — safe by construction, no collision possible between sessions.

Shared .git history

Independent working dirs

Normal PR on finish



Sandbox Containment

Cloud: fully isolated compute.
Local: bounded scope, admin-configurable. No lateral movement.

Cloud: isolated compute

Local: bounded scope

Admin-configured access



Policy Control

Org admins configure sandbox types, Agent Merge eligibility, and audit scope per repo.

Sandbox type policy

Merge eligibility rules

Full audit log

worktree-init.sh — Pre-Initialize for Fleet Sessions

worktree-init.sh

```
#!/usr/bin/env bash
# Usage: ./worktree-init.sh <repo> <n> [prefix]
set -euo pipefail
REPO_PATH="${1:?}"
N_AGENTS="${2:?}"
PREFIX="${3:-agent}"
TIMESTAMP=$(date +%Y%m%d-%H%M)
cd "$REPO_PATH"
git fetch origin main --quiet
for i in $(seq 1 "$N_AGENTS"); do
    BRANCH="${PREFIX}-${TIMESTAMP}-${i}"
    DIR=".. /$(basename "$REPO_PATH")-${PREFIX}-${i}"
    git worktree add -b "$BRANCH" "$DIR" origin/main
    echo "  ✓ Worktree $i: $DIR"
done
```

One branch per agent

Each worktree gets its own branch from origin/main

Independent directories

Agents cannot overwrite each other's working files

Normal PRs on finish

Each session branch closes as a standard pull request

Control Surfaces

Autonomy is a dial, not a switch — five slides to calibrate and control the fleet



Three Modes

Interactive → Plan → Autopilot,
matched to task risk



Canvas + Handbrake

Shift to Interactive mid-session without
losing the worktree



Issue Brief

Issue quality = agent quality — the
upstream lever

Autonomy is a dial, not a switch —

↳ Interactive, Plan, and Autopilot give the right trust level per task

Autonomy Is a Dial, Not a Switch



Interactive

Agent proposes each step and waits for approval. Best for exploratory or ambiguous tasks.

Step-by-step approval

Transparent work log

Best: unclear approach



Plan

Agent writes a complete plan; human reviews and approves before any code changes begin.

Full plan before code

Approve before execute

Best: scope clear, plan wanted



Autopilot

Agent executes end-to-end, opens a PR when done. Best for routine well-specified tasks.

Fire and review PR

Low blast radius tasks

EM policy controls scope



EM Policy: sandbox types · Agent Merge eligibility per repo · audit log with session ID, user, timestamp

autonomy-mode-guide.md — Take-Home Artifact #1

Match task characteristics to mode — the CVE row connects directly to S4

Exploratory	Approach unclear → agent proposes, human approves each step	Interactive
Well-Scoped	Scope clear, plan review wanted — e.g., OAuth2 feature add	Plan
CVE/Security	High blast radius: Plan mode analyzes usages, drafts plan first	Plan
Routine	Known pattern, low blast radius — dep updates, tests, triage	Autopilot
New Pattern	Untested task type — run Interactive or Plan first	Interactive

If the approach is non-obvious, default to Plan mode — scope mismatches become PR surprises in Autopilot

Canvas — Coordinate, Steer, and Inspect the Fleet



What a Canvas Contains

A live bidirectional workspace — plan, PR diff, terminal stream, CI state.

Plan or task brief

PR diff view (real-time)

Terminal stream

CI + deployment state



The Handbrake

Pull the handbrake: shift Autopilot to Interactive mid-session without canceling the worktree.

Shift to Interactive mid-session

Add context notes to steer

MCP: external tool access

Cancel cleanly — no commits



Operator Controls

Visibility and navigation across the fleet — no new autonomy modes.

Auto model attribution per request

Credit/cache details when available

Direct entry into shared sessions

/side explores without disrupting main

Issue Quality = Agent Quality

issue-brief-template.md

Objective

One paragraph: what we want achieved and why.
Be specific – agents parse this as primary context.

Acceptance Criteria

- [] Criterion 1 – concrete and testable
- [] Criterion 2 – concrete and testable

Scope

****In scope:**** explicit file paths or modules
****Out of scope:**** explicit exclusions

Autonomy Mode

- [] Interactive
- [] Plan
- [x] Autopilot

Objective section

Agents parse this as primary context before execution begins

Acceptance criteria

Binary, verifiable conditions — agents know when they're done

Scope exclusions

Explicit out-of-scope prevents surprise file edits

3-minute investment

Three minutes in the brief is the upstream determinant of output quality

What Breaks — Three Causal Failure Modes



Autopilot on Ambiguous Tasks

Unclear scope → decisions you'd want to intercept. Scope mismatches become PR surprises.

Use Plan when approach unclear

Ambiguity → surprise PRs



Unstructured Issue Brief

No acceptance criteria → two-sentence PR regardless of mode.
Issue quality = agent quality.

Invest 3 min in the brief

Issue quality = agent quality



Skipping Canvas Escalation

No recovery path when the agent veers — Canvas handbrake is the only mid-course correction.

Open Canvas for Autopilot

Pull the handbrake early

Sessions to Systems

The CVE payoff, workflow anatomy, and the Monday dispatch that runs itself



CVE Response

Plan mode, two agents, patch and tests
in 20 minutes



Workflow Anatomy

agent-workflow.yml structure and
Agent Merge lifecycle



Monday Dispatch

The proven session pattern that now
runs automatically

CVE in session management:

↳ Plan mode • two agents • PR ready in 20 min • full audit trail

CVE Response — Plan Mode, Two-Agent Sequence

High-stakes autonomous work with a full audit trail

✗ Without the Fleet Model

- 1 CVE flagged
Engineer pulls from main and starts investigation
- 2 Manual impact analysis
Dev reads every usage of affected package — 30–90 min
- 3 Patch and test manually
Careful edits to avoid regressions — context-heavy work
- 4 PR ready
2–3 hours after CVE flagged

 2–3 hours — manual context switching

✓ With Plan Mode + Two Agents

- 1 CVE flagged
Agent 1 dispatched in Plan mode immediately
- 2 Agent 1: impact analysis
Analyzes all usages, drafts remediation plan for review
- 3 Agent 2: patch + tests
After plan approved — upgrade and regression tests in Autopilot
- 4 PR + audit trail ready
~20 min — patch, tests, full agent action log

✓ ~20 min — before incident response call ends

2–3 hours → 20 min · full audit trail for security review

agent-workflow.yml + Agent Merge Lifecycle

agent-workflow.yml

```
name: Weekly Dependency Update
trigger:
  schedule:
    cron: "0 9 * * MON"
  manual: true

session:
  mode: autopilot
  environment: cloud-sandbox
  branch_prefix: "agent/deps-update"

task: |
  Update all dev dependencies to latest minor
  versions. Run npm test and npm run lint.
  If any test fails, revert that package.

pr:
  title: "chore: weekly dev dependency update"
  labels: [dependencies, automated]
  auto_merge: true
  merge_conditions:
```



Schedule + manual trigger

Runs every Monday 09:00 UTC automatically;
also dispatchable manually



Agent Merge lifecycle

Agent monitors CI, handles reviewer feedback,
merges when conditions are met



Versioned in repo

Workflow definitions are auditable and rollback-
able like any config change



Human in the loop

Keep human merge step for novel features,
security paths, or uncertain-scope sessions

The Monday Dispatch Now Runs Itself

Before

Manual dispatch every Monday
Tech lead triggers each task individually

One agent at a time
Sequential work — throughput capped at one

CVE response: 2–3 hours
Manual impact analysis before any patching

Recurring patterns repeated manually
No way to save and rerun a session

After

Workflows trigger automatically
Dep updates run every Monday at 09:00 UTC

Five agents, five issues
Fleet dispatch surfaces completed work in parallel

CVE response: ~20 minutes
Plan mode + two agents + audit trail, ready to review

Sessions saved as workflows
Proven patterns become permanent automation library

5×

throughput on routine tasks

~20 min

CVE patch + regression tests

0 manual

triggers for recurring workflows

From Serial Bottleneck to Fleet Coordination

Before

One Copilot session at a time — throughput capped at one

Manual context switching between three open terminal sessions

CVE response: 2–3 hours of manual impact analysis before patching

Recurring tasks manually triggered every time they come up

After

Five agents, five issues, zero conflicts — fleet dispatched in parallel

My Work dashboard surfaces all sessions, PRs, and CI in one view

CVE response: ~20 min — Plan mode, two agents, full audit trail

Proven session patterns converted to scheduled, event-driven workflows

5×

throughput on routine and well-scoped backlog work

20 min

CVE patch and regression tests — before the incident call ends

0 collisions

five parallel agents, isolated worktrees, safe by design

✓ What You Can Do Today

Today

- ❑ Download the Copilot App from github.com/features/ai/github-app
- ❑ Sign in (Pro, Pro+, Business, or Enterprise required)
- ❑ Explore My Work — sessions, PRs, automations in one view
- ❑ Start one Plan-mode session on a good-first-agent-task issue

This Week

- ❑ Run `worktree-init.sh` to pre-initialize two parallel worktrees
- ❑ Dispatch two agents — one Plan, one Autopilot — to independent issues
- ❑ Review both PRs and compare how each session logged its reasoning
- ❑ Draft your first issue-brief-template.md and measure autonomy quality

This Month

- ❑ Audit the backlog for agent-ready tickets with clear acceptance criteria
- ❑ Configure a first repeatable workflow for a recurring maintenance task
- ❑ Test Agent Merge on a low-risk automated workflow
- ❑ Share `autonomy-mode-guide.md` with the team and align on mode norms

🔑 Key Takeaway

The Monday dispatch that took 2 hours manually now runs itself — start with one issue today.

 Official Documentation[About the GitHub Copilot app — GitHub Docs](#)

Core concepts, autonomy modes, policy configuration, and how the app differs from VS Code

[Getting started with the GitHub Copilot app — GitHub Docs](#)

Prerequisites, installation steps, and first session walkthrough

 Related Tech Talks[Agent Teams](#)

Architectural patterns behind multi-agent systems the Copilot App makes accessible without custom code

[Agentic Workflows](#)

Designing trigger and automation patterns for GitHub Actions and Copilot together

 Blog Posts & Announcements[GitHub Copilot app: The agent-native desktop experience — GitHub Blog](#)

Launch announcement with Canvas and Agent Merge detail, and design rationale

[GitHub Copilot weekly releases - August 3: Copilot App controls](#)

Auto model attribution, credit/cache details, /side exploration, and session navigation

[GitHub Copilot app product page](#)

Download, subscription access, and feature summary



The GitHub Copilot App

Running a Fleet of Agents — From one collaborator to a coordinated fleet

Five agents, zero conflicts

Isolated worktrees make parallel work safe by design, not coordination

Autonomy is a dial

Interactive → Plan → Autopilot — the right trust level per task

Pull the handbrake

Canvas shifts Autopilot to Interactive mid-session without losing the worktree

Issue quality = agent quality

Three minutes in the brief is the upstream lever for autonomous output

Which task on your backlog is most ready for the fleet model right now?